Week3 - Day2 Assignment - JIRA Integration Test Case Generator Tool

Requirements Implemented:

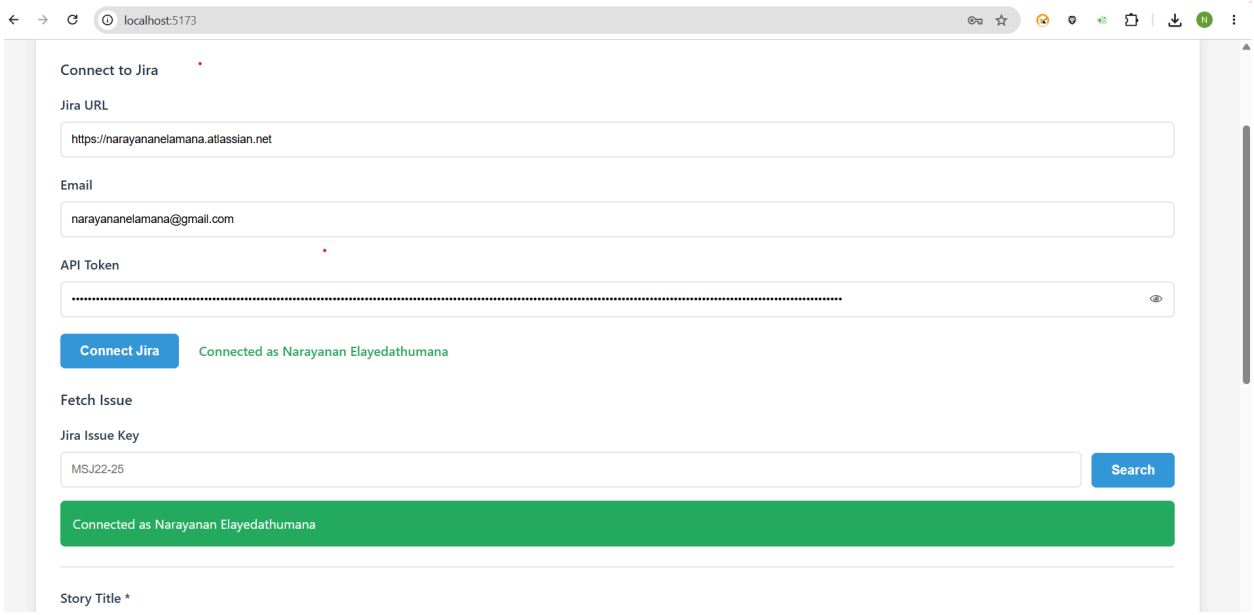
Connect to Jira section:

- Provide the following input fields:
    - Jira URL
    - Email Address
    - API Token
- Allow users to enter their Jira credentials and connect the Test Case Generator tool to their Jira account.
- Display a success message when the connection is established successfully.
- Display an appropriate error message if the connection fails due to invalid credentials or configuration.
- Provide an eye icon to show or hide the API Token for user convenience.

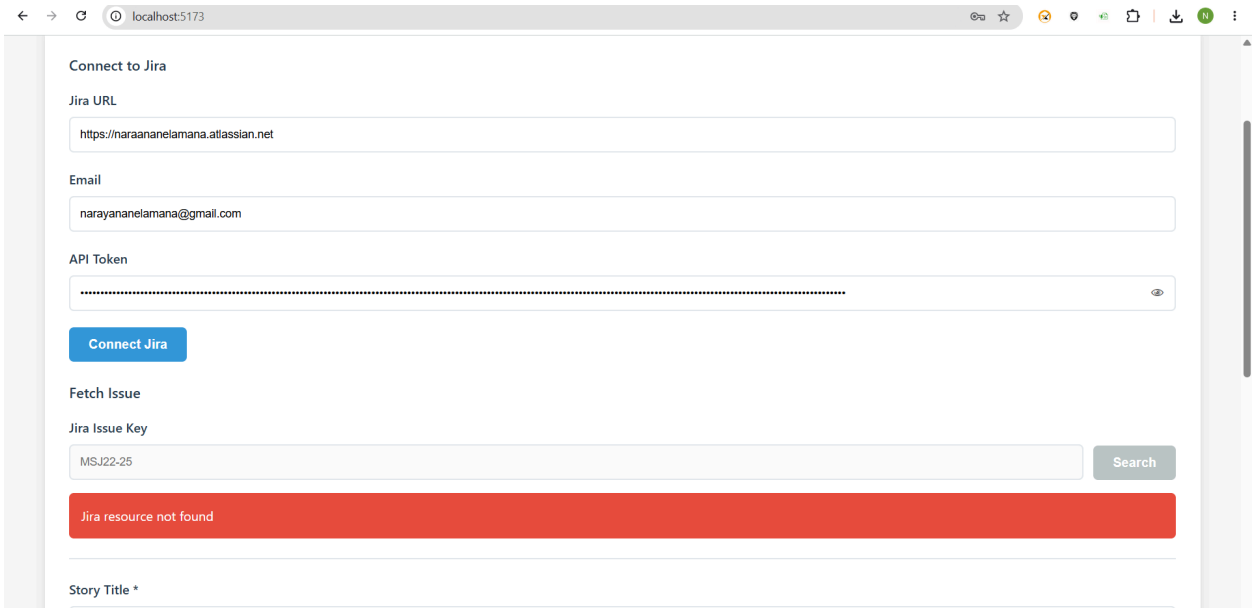
Fetch Jira Issue section:

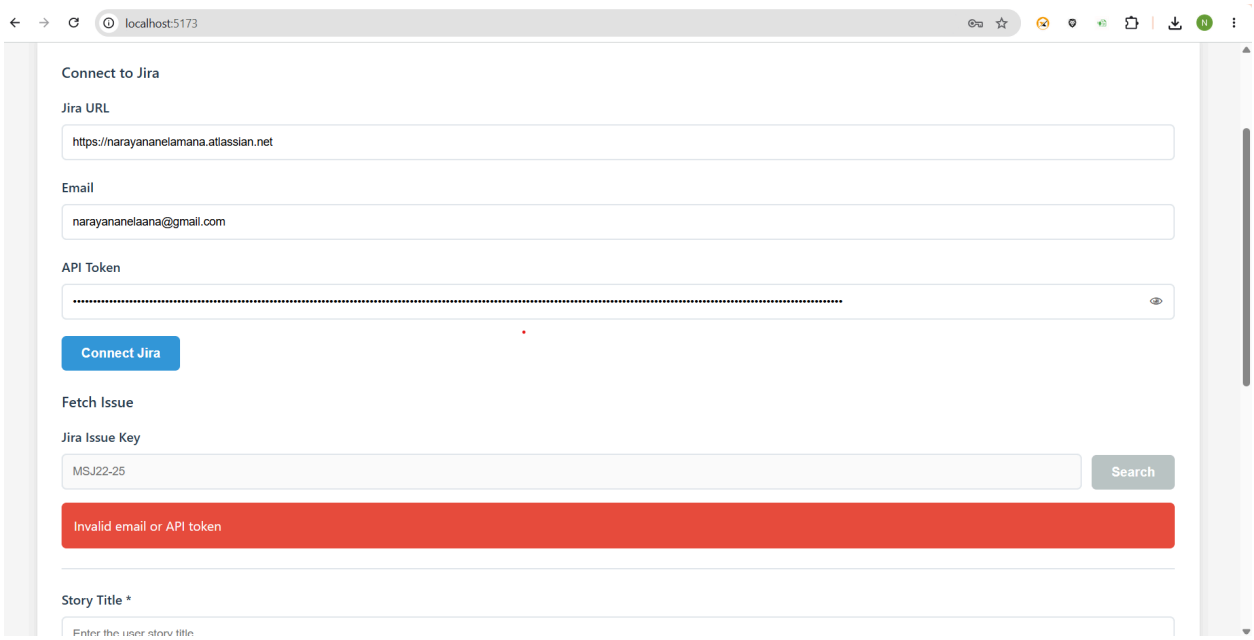
- Provide a Jira Issue Key input field.
- Provide a Search button to fetch issue details.
- After a successful Jira connection, users can enter a valid Jira Issue Key and search for the issue.
- On successful retrieval, automatically populate the Test Case Generator fields with the Jira issue details.
- Display a success message when the issue is fetched successfully.
- Display an appropriate error message if the entered Jira Issue Key is invalid or the issue cannot be found.

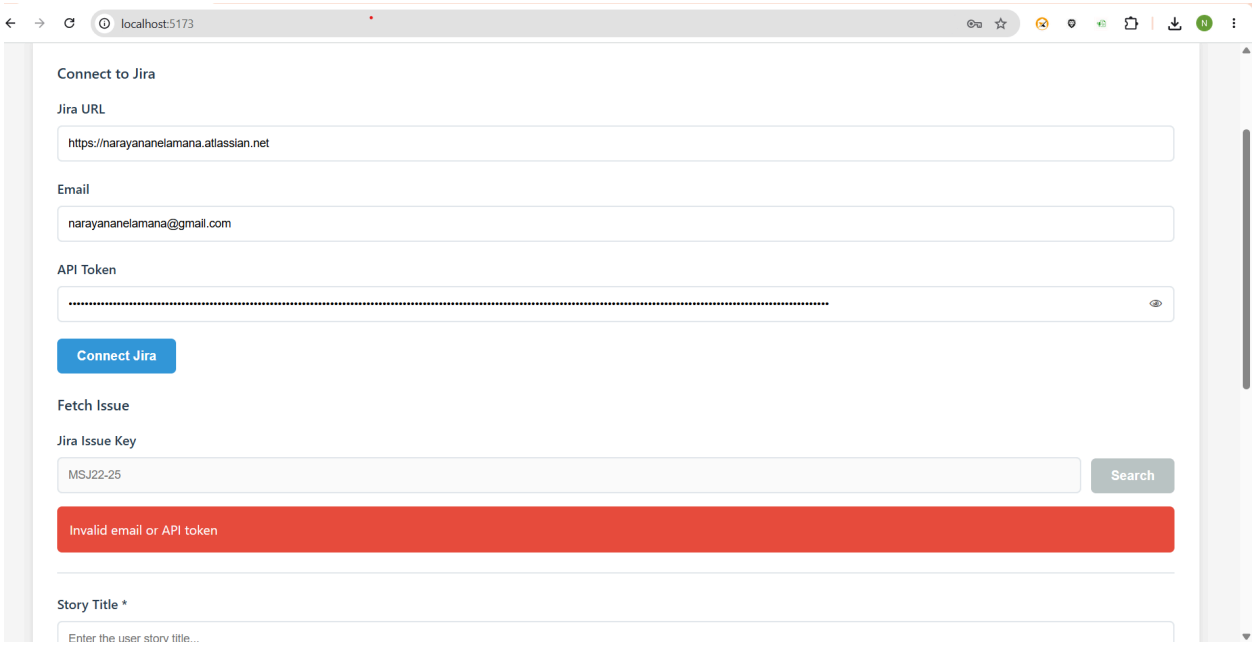
Connect to JIRA account Flow:



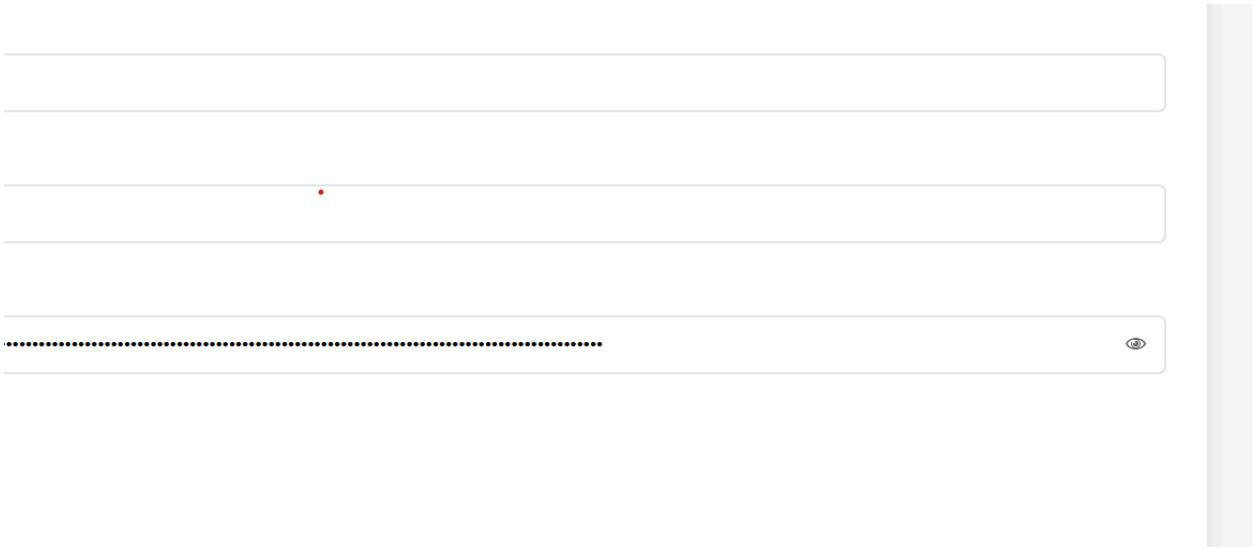
Error Handling:

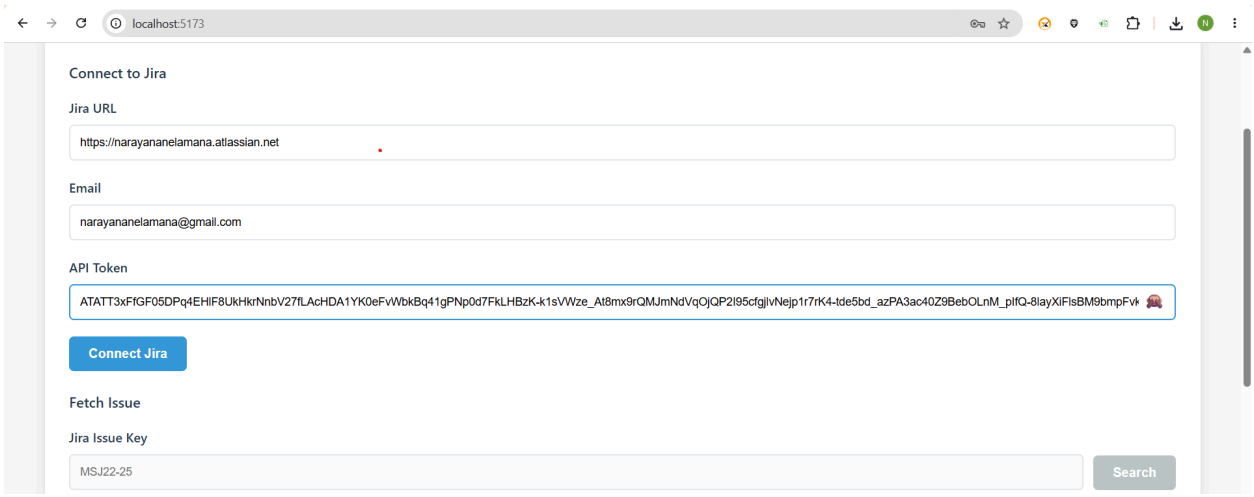



localhost:5173
Connect to Jira
Jira URL
https://narayananelamana.atlassian.net
Email
narayananelaana@gmail.com
API Token
Connect Jira
Fetch Issue
Jira Issue Key
MSJ22-25
Search
Invalid email or API token
Story Title *

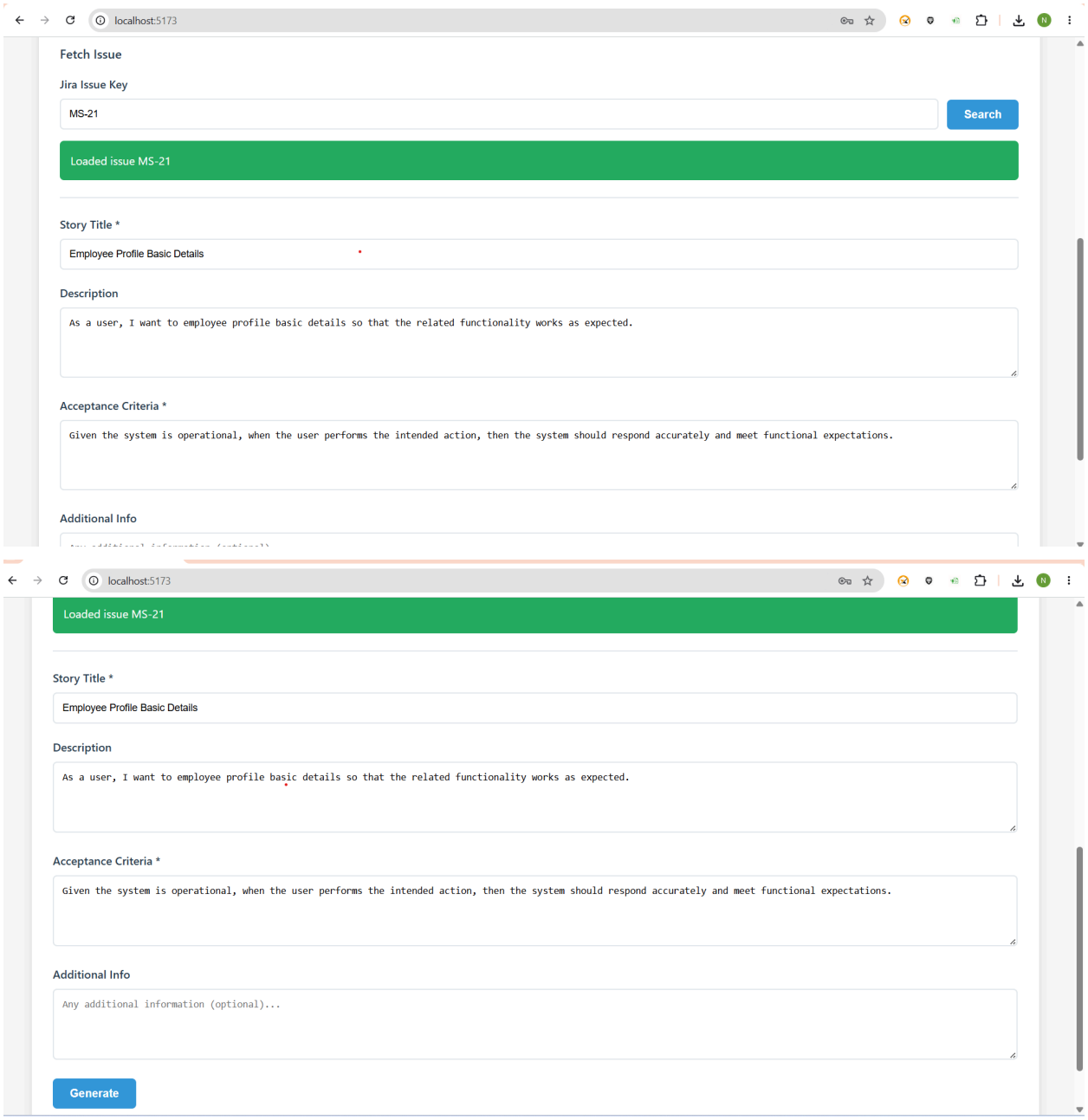

localhost:5173
Connect to Jira
Jira URL
https://narayananelamana.atlassian.net
Email
narayananelamana@gmail.com
API Token
Connect Jira
Fetch Issue
Jira Issue Key
MSJ22-25
Search
Invalid email or API token
Story Title *
Enter the user story title...

Reveal and hide API Token:



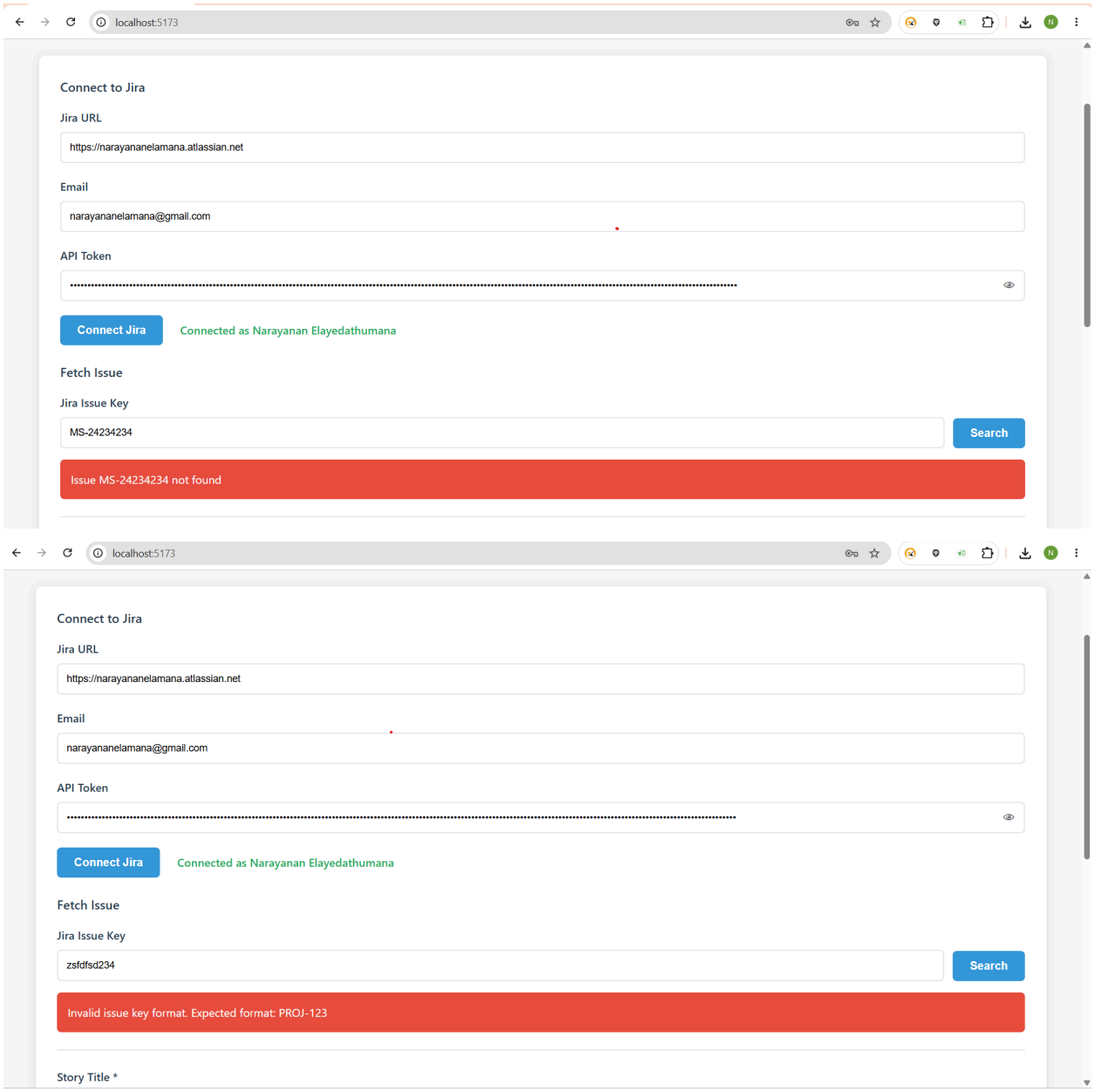


Fetch Jira Issue flow:

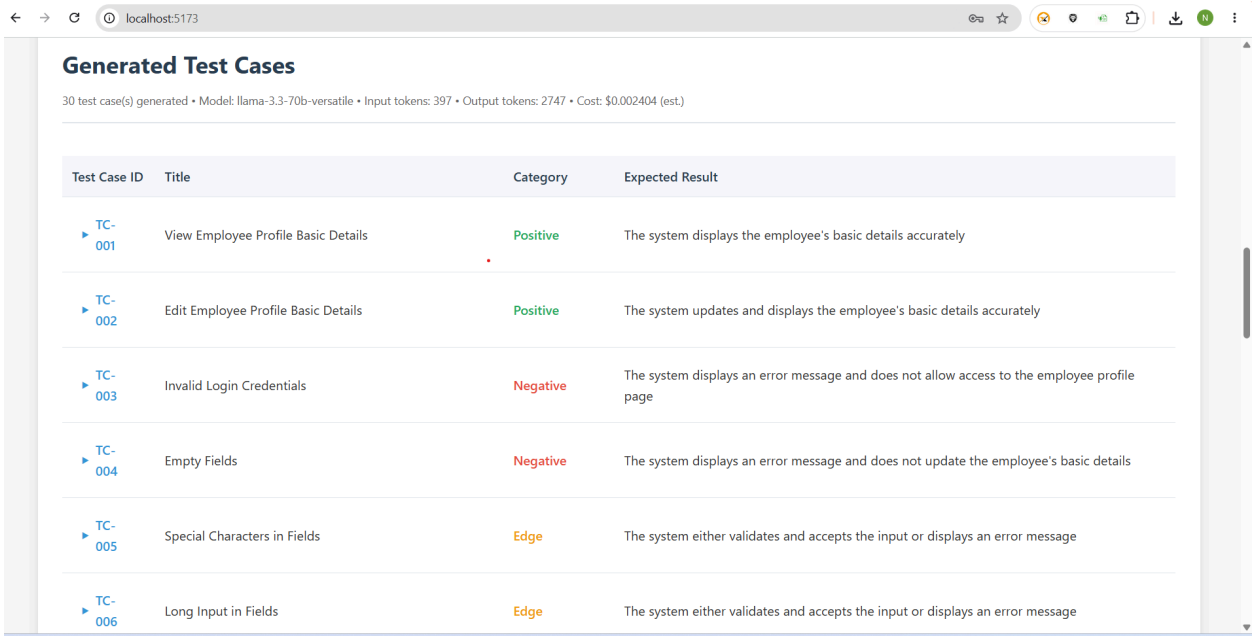


Error Handling:

Invalid JIRA ISSUE ID:



Existing Feature of Test Case Generation works fine (after auto-population of the fields directly from JIRA)



AI Tool Details:

AI Tool Used for Vibe Coding : Cursor
LLM Model : Auto

Initial Prompt Used for Making the phased plan(Ask mode):

Background

The existing application is an AI-powered Test Case Generator.

Current Functionality

The application currently allows users to enter:

- **Story Title**
- **Description**
- **Acceptance Criteria**
- **Additional Information**

When the user clicks the Generate button:

1. **The frontend sends the entered data to the backend.**
2. **The backend calls the Groq LLM API using the selected model.**
3. **The LLM generates detailed test cases.**
4. **The generated test cases are displayed in the frontend.**

Please thoroughly analyze the entire existing codebase, including both frontend and backend implementations, before proposing any changes.

New Feature Requirement: Jira Integration

Objective

Integrate Jira with the existing application so users can fetch User Story details directly from Jira and automatically populate the existing form fields.

Functional Requirements

Phase 1 - Jira Connection

Connect Jira Button

Add a new button:

"Connect Jira"

When clicked:

- **Establish a connection with Jira using Jira REST APIs.**
- **Follow industry-standard authentication practices.**
- **Design the solution so credentials are configurable through environment variables.**
- **Do not hardcode credentials in source code.**

Expected credentials:

- **Jira Organization URL**
- **Jira Username / Email**

- **Jira API Token**

I will provide these credentials later for testing.

Phase 2 - Search Jira Issue

After successful Jira connection:

UI Changes

Add:

- **Jira Issue Key textbox**
- **Search icon/button**

Example Issue Keys:

- **MSJ22-25**
- **PROJ-123**
- **TEST-456**

Search Workflow

When user enters an Issue Key and clicks Search:

1. **Validate the issue key.**
2. **Display loading indicator/spinner.**
3. **Call Jira REST API.**
4. **Fetch issue details.**
5. **Handle all error scenarios gracefully.**

Examples:

- **Invalid issue key**
- **Authentication failure**
- **Jira unavailable**
- **Network timeout**
- **Permission denied**
- **Empty response**

Data Mapping

Populate existing fields with Jira data:

Existing Field	Jira Source
Story Title	Summary
Description	Description
Acceptance Criteria	Acceptance Criteria field
Additional Info	Any additional Jira metadata if available

If Acceptance Criteria is stored in a custom field, identify and document the required Jira field mapping.

User Experience Requirements

Loading State

While fetching Jira data:

- **Disable search button**
- **Show spinner/loader**
- **Display status message**

Success State

- **Populate fields automatically**
- **Show success notification**

Error State

- **Show user-friendly error message**
- **Preserve existing user-entered data**

Backend Requirements

Analyze existing backend architecture and implement:

Jira Service Layer

Create:

- **Jira API Client**
- **Authentication Module**
- **Issue Fetch Service**
- **Error Handling Layer**

API Endpoints

Design and implement the required backend endpoints for:

- **Jira Connection Validation**
- **Jira Issue Retrieval**

Follow the existing coding patterns and architecture already used in the project.

Frontend Requirements

Analyze existing frontend architecture and implement:

Components

Create or modify components necessary for:

- **Connect Jira button**
- **Jira issue search input**
- **Search button**
- **Loading indicator**
- **Success/Error notifications**

Follow the current UI design and coding standards already present in the application.

Technical Analysis Required

Before implementing any code:

Provide a detailed analysis of:

1. **Existing frontend architecture**
2. **Existing backend architecture**
3. **Current API flow**
4. **Groq integration flow**
5. **Proposed Jira integration architecture**
6. **Files that need modification**
7. **New files that need to be created**
8. **Environment variables required**
9. **Security considerations**
10. **Potential risks and limitations**

Implementation Plan

Break the implementation into multiple phases.

For each phase provide:

- **Objective**
- **Files to be modified**
- **Files to be created**
- **Backend changes**
- **Frontend changes**
- **Testing strategy**
- **Rollback considerations**

Do not start coding immediately.

First perform a complete workspace analysis and provide a detailed implementation plan.

Architecture Review

As a Senior Architect and Full-Stack Engineer with 10+ years of experience:

Please review whether Jira integration is the best approach.

Also suggest any better alternatives or enhancements, such as:

- **Jira OAuth 2.0 authentication**
- **Personal API Token authentication**
- **Jira Cloud App approach**
- **Caching strategy**
- **Recently searched issues**
- **Auto-complete issue search**
- **Project-level issue browsing**
- **Bulk story import**
- **Fetching linked stories/subtasks**
- **Generating test cases directly from Jira issues**

Explain the pros and cons of each approach.

Deliverables

Before writing code, provide:

1. **Complete architecture analysis**
2. **File-level impact analysis**
3. **Implementation phases**
4. **Security recommendations**
5. **Best-practice Jira integration approach**
6. **Detailed execution plan**

Only after approval should implementation begin.